Intelligent Energy Management Systems: Optimizing Energy Consumption through AI

By Adam Ben Rhaiem - University: SUP'COM - Company: ACTIA ES

Introduction

Event detection in NILM refers to the identification of specific occurrences (events) in the power signal that correspond to changes in appliance behavior. These events might include:

- **Turn-on events**: When an appliance starts consuming power.
- **Turn-off events**: When an appliance stops consuming power.
- **State changes**: When an appliance switches between different operational states (e.g., from idle to active, or from heating to cooling).

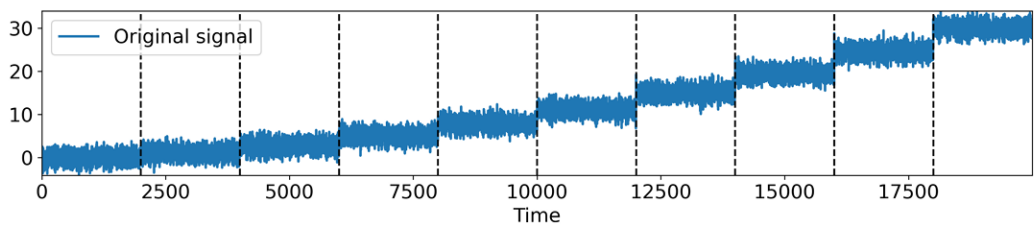
Event detection in NILM typically involves analyzing power consumption data (voltage, current, or aggregated power) to:

- Detect transitions in appliance operation.
- Segment the data into meaningful time periods that correspond to different appliance activities.
- Estimate when appliances are operating, idle, or transitioning between states.

Change Point Detection

Definition

Change Point is a moment of time when a time series changes its behaviour.



Problem:

detect all change points in the time series. Labels are unknown.

Change point detectors

Change point detectors are algorithms designed to detect change points in streaming or sequential observations. These detectors analyze the data stream and identify points where the underlying data distribution has changed significantly.

ruptures Library for change point detection

ruptures is a Python library for off-line change point detection. This package provides methods for the analysis and segmentation of non-stationary signals. Implemented algorithms include exact and approximate detection for various parametric and non-parametric models. ruptures focuses on ease of use by providing a well-documented and consistent interface. In addition, thanks to its modular structure, different algorithms and models can be connected and extended within this package.

1. CUSUM (Cumulative Sum Control Chart)

How it works:

- **Define Parameters**:
  - **Reference Value** ($\mu 0$) : The expected mean or baseline value of the data.
  - **Sensitivity** (k): Determines how sensitive the system is to small shifts.
  - **Threshold** (h): The value at which a significant change is flagged.
- **Monitor Data**:
  - For each data point xt, calculate the cumulative sum of deviations from the reference value.
- **Detect Changes**:
  - If the cumulative sum exceeds the predefined threshold (h), an event (change) is detected.
- **Reset After Detection**:
  - After an event is detected, reset the cumulative sum to **zero** and continue monitoring.

Strengths:

- **Effective for small changes**: Can detect subtle shifts in data trends.
- **Low computational cost**: Efficient for real-time applications.

Performance Metrics

- **Accuracy**: High, when tuned properly.
- **Speed**: Fast (low computational cost).
- **False Alarm Rate**: Moderate, depends on noise and tuning of kkk.
- **Robustness**: Can struggle with highly fluctuating signals.

Limitations:

- **Sensitive to noise**: Requires preprocessing (e.g., smoothing, filtering).
- **Parameter tuning required**: Selection of control limits and reference values affects performance.
- **Not suitable for abrupt changes**: Works better for gradual drifts rather than sudden spikes.

2. Constant False Alarm Rate (CFAR)

How it works:

- Defines a sliding window around the test cell and its guard cells.
- Computes a noise estimate from surrounding cells.
- Sets a detection threshold that adapts to the environment.

The CFARDetector object implements a one-dimensional constant false-alarm rate (CFAR) detector. Detection processing is performed on selected elements (called cells) of the input data. A detection is declared when a cell value in the input data exceeds a threshold. To maintain a constant false alarm-rate, the threshold is set to a multiple of the local noise power of the input data. The detector estimates local noise power for a cell-under-test *(CUT)* from surrounding cells using one of three cell averaging methods, or an order statistics method. The cell-averaging methods are cell averaging (CA), greatest-of cell averaging (GOCA), or smallest-of cell averaging (SOCA).

Problem	CA-CFAR	GO-CFAR	SO-CFAR	OS-CFAR
Clutter Edges	Poor	Good	Poor	Good
Interfering Targets	Poor	Poor	Good	Good

For each test cell, the detector:

1. Estimates the noise statistic from the cell values in the training band surrounding the CUT cell.
2. Computes the threshold by multiplying the noise estimate by the threshold factor.
3. Compares the CUT cell value to the threshold to determine whether a target is present or absent. If the value is greater than the threshold, a target is present.

detector = phased.CFARDetector creates a CFAR detector System object™, detector. The object performs CFAR detection on input data.

To run the detector:

1. Create the phased.CFARDetector object and set its properties.
2. Call the object with arguments, as if it were a function.

Strengths:

- **Robust in noisy environments:** CFAR algorithms are specifically designed to maintain a constant false alarm rate in the presence of varying noise and clutter levels, making them highly effective in high-noise applications such as radar and sonar.
- **Adaptive thresholding:** CFAR dynamically adjusts detection thresholds based on local noise estimates, allowing for consistent performance despite

Limitations:

- **Parameter selection is crucial**: Requires careful tuning of window sizes and guard cells.
- **High false alarm rate in cluttered environments**: Performance degrades in non-uniform noise.

Performance Metrics

- **Accuracy**: Very good in noisy or complex environments.
- **Speed**: Moderate, can be slower compared to CUSUM due to adaptive thresholding.
- **False Alarm Rate**: Low, if parameters are set properly.
- **Robustness**: High against noise and clutter.

3. BinSeg (Binary Segmentation)

How it works:

- Finds the point that maximizes the change in variance or mean.
- Splits the data at that point and applies the same process recursively.
- Stops when no significant changes are detected.

Strengths:

- **Efficient for multiple change points**: Recursively segments the data, unlike simpler methods.
- **Global approach**: Considers the entire dataset rather than only local statistics.

Limitations:

- **Computationally expensive**: Recursive segmentation is costly for large datasets.
- **Error propagation :** Incorrect early splits can affect later segmentations.

Performance Metrics

- **Accuracy**: High, especially when changes are sharp and well-defined.
- **Speed**: Moderate to high but can become slower with large datasets due to iterative segmentation.
- **False Alarm Rate**: Moderate but can be reduced with correct tuning of the stopping rule.
- **Robustness**: Fairly robust but prone to overfitting in noisy data.

4. Window Sliding Segmentation

How it works :

- The algorithm uses two windows which slide along the data stream. The statistical properties of the signals within each window are compared with a discrepancy measure. For a given cost function $c(\cdot)$, a discrepancy measure a discrepancy measure is derived $d(\cdot,\cdot)$ as follows:

$$d(y_{u..v}, y_{v..w}) = c(y_{u..w}) - c(y_{u..v}) - c(y_{v..w})$$

Strengths:

- **Less prone to error propagation**: Since it looks at local segments rather than making global splits.
- **Useful for short-term event detection**: Works well in speech processing and anomaly detection in IoT.

Limitations:

- **Window size selection is critical**: Choosing the wrong window size can lead to missing events or excessive noise.
- **Not effective for long-range dependencies**: Cannot detect patterns beyond the window length.

Performance Metrics

- **Accuracy**: Good, especially for local changes. Struggles with detecting long-term trends or sudden, large shifts.
- **Speed**: Moderate, depending on window size and the complexity of the discrepancy measure.
- **False Alarm Rate**: Relatively low but depends on window size and threshold settings.
- **Robustness**: Moderate, depends heavily on the sliding window settings and the measure used.

Conclusion

Each algorithm has its strengths and weaknesses based on the type of event detection required and the nature of the data. For example, **CUSUM** excels in detecting small shifts with low computational cost, whereas **CFAR** is more robust in noisy environments but computationally heavier. **Binseg** is highly accurate but may overfit, while **window sliding segmentation** is simple and flexible but struggles with larger shifts.